

# Tutorial de Programación

## Comparación entre Python y C++

Este tutorial presenta una comparación detallada entre los lenguajes de programación Python y C++, incluyendo ejemplos prácticos, medición de tiempos de ejecución, manejo de errores y técnicas de depuración.

### Contenido del Tutorial:

- Fase 1: Implementación de contador regresivo
- Fase 2: Detección y corrección de errores
- Comparación de rendimiento entre lenguajes
- Análisis de mensajes de error y depuración

### Proyecto JAM - Tutorial Educativo

Corporación Uniremington, Sede Cali

Fecha de creación: 13 de September de 2025

# Índice

## **1. Fase 1: Contador Regresivo**

- 1.1 Implementación en Python
- 1.2 Implementación en C++
- 1.3 Comparación de tiempos de ejecución
- 1.4 Análisis de resultados

## **2. Fase 2: Corrección de Errores**

- 2.1 Código Python con errores
- 2.2 Código Python corregido
- 2.3 Código C++ con errores
- 2.4 Código C++ corregido
- 2.5 Análisis de detección de errores
- 2.6 Interpretación de mensajes de error

# Fase 1: Contador Regresivo

En esta fase implementaremos un contador regresivo en ambos lenguajes para comparar la sintaxis, facilidad de implementación y tiempo de ejecución.

## 1.1 Implementación en Python

El código Python utiliza la librería *time* para medir el tiempo de ejecución y crear pausas. La sintaxis es simple y legible, con tipado dinámico.

| Código | Python                                                                                       |
|--------|----------------------------------------------------------------------------------------------|
| 1      | <code>#!/usr/bin/env python3</code>                                                          |
| 2      | <code>"""</code>                                                                             |
| 3      | <code>Fase 1: Contador regresivo en Python</code>                                            |
| 4      | <code>Tutorial de programación - Comparación Python vs C++</code>                            |
| 5      | <code>"""</code>                                                                             |
| 6      |                                                                                              |
| 7      | <code>import time # Importa la librería time para medir tiempo y pausas</code>               |
| 8      |                                                                                              |
| 9      | <code>def contador_regresivo(numero_inicial):</code>                                         |
| 10     | <code>    """</code>                                                                         |
| 11     | <code>    Función que implementa un contador regresivo desde un número dado hasta 0</code>   |
| 12     |                                                                                              |
| 13     | <code>    Args:</code>                                                                       |
| 14     | <code>        numero_inicial (int): El número desde donde empezar la cuenta regresiva</code> |
| 15     | <code>    """</code>                                                                         |
| 16     | <code>    print(f"Iniciando contador regresivo desde {numero_inicial}")</code>               |
| 17     |                                                                                              |
| 18     | <code>    # Ciclo while que continúa mientras el número sea mayor a 0</code>                 |
| 19     | <code>    while numero_inicial &amp;gt; 0:</code>                                            |
| 20     | <code>        print(f"Cuenta regresiva: {numero_inicial}") # Muestra el número actual</code> |
| 21     | <code>        numero_inicial -= 1 # Decrementa el contador en 1</code>                       |
| 22     | <code>        time.sleep(0.1) # Pausa de 0.1 segundos para visualizar mejor el conteo</code> |
| 23     |                                                                                              |
| 24     | <code>    print("¡Cuenta regresiva terminada! ■")</code>                                     |
| 25     |                                                                                              |
| 26     | <code>def main():</code>                                                                     |
| 27     | <code>    """</code>                                                                         |
| 28     | <code>    Función principal que ejecuta el programa y mide el tiempo de ejecución</code>     |
| 29     | <code>    """</code>                                                                         |
| 30     | <code>    # Registra el tiempo de inicio</code>                                              |
| 31     | <code>    tiempo_inicio = time.time()</code>                                                 |

|    |                                                                    |
|----|--------------------------------------------------------------------|
| 32 |                                                                    |
| 33 | # Ejecuta el contador regresivo desde 10                           |
| 34 | contador_regresivo(10)                                             |
| 35 |                                                                    |
| 36 | # Registra el tiempo de finalización                               |
| 37 | tiempo_fin = time.time()                                           |
| 38 |                                                                    |
| 39 | # Calcula y muestra el tiempo total de ejecución                   |
| 40 | tiempo_total = tiempo_fin - tiempo_inicio                          |
| 41 | print(f"\nTiempo total de ejecución: {tiempo_total:.4f} segundos") |
| 42 |                                                                    |
| 43 | # Verifica si este archivo se está ejecutando directamente         |
| 44 | if __name__ == "__main__":                                         |
| 45 | main()                                                             |

## 1.2 Implementación en C++

El código C++ utiliza las librerías *chrono* y *thread* para medir tiempo y crear pausas. Requiere más declaraciones explícitas pero ofrece mayor control.

| Código | C++                                                                                 |
|--------|-------------------------------------------------------------------------------------|
| 1      | #include <iostream> // Para operaciones de entrada y salida (cout, cin)             |
| 2      | #include <chrono> // Para medir tiempo de ejecución                                 |
| 3      | #include <thread> // Para pausas en la ejecución (sleep)                            |
| 4      |                                                                                     |
| 5      | using namespace std; // Permite usar cout, cin, etc. sin std::                      |
| 6      |                                                                                     |
| 7      | /**                                                                                 |
| 8      | * Función que implementa un contador regresivo desde un número dado hasta 0         |
| 9      | *                                                                                   |
| 10     | * @param numero_inicial El número desde donde empezar la cuenta regresiva           |
| 11     | */                                                                                  |
| 12     | void contadorRegresivo(int numero_inicial) {                                        |
| 13     | cout <<< "Iniciando contador regresivo desde " << numero_inicial <<< "              |
| 14     |                                                                                     |
| 15     | // Ciclo while que continúa mientras el número sea mayor a 0                        |
| 16     | while (numero_inicial > 0) {                                                        |
| 17     | cout <<< "Cuenta regresiva: " << numero_inicial <<< endl; //                        |
| 18     | numero_inicial--; // Decrementa el contador en 1                                    |
| 19     | this_thread::sleep_for(chrono::milliseconds(100)); // Pausa de 100ms (0.1 segundos) |
| 20     | }                                                                                   |
| 21     |                                                                                     |

|    |                                                                                          |
|----|------------------------------------------------------------------------------------------|
| 22 | cout &lt;&lt; "¡Cuenta regresiva terminada! ■" &lt;&lt; endl;                            |
| 23 | }                                                                                        |
| 24 |                                                                                          |
| 25 | /**                                                                                      |
| 26 | * Función principal que ejecuta el programa y mide el tiempo de ejecución                |
| 27 | */                                                                                       |
| 28 | int main() {                                                                             |
| 29 | // Registra el tiempo de inicio usando chrono de alta resolución                         |
| 30 | auto tiempo_inicio = chrono::high_resolution_clock::now();                               |
| 31 |                                                                                          |
| 32 | // Ejecuta el contador regresivo desde 10                                                |
| 33 | contadorRegresivo(10);                                                                   |
| 34 |                                                                                          |
| 35 | // Registra el tiempo de finalización                                                    |
| 36 | auto tiempo_fin = chrono::high_resolution_clock::now();                                  |
| 37 |                                                                                          |
| 38 | // Calcula la duración total en milisegundos                                             |
| 39 | auto duracion = chrono::duration_cast<chrono::milliseconds>(tiempo_fin - tiempo_inicio); |
| 40 |                                                                                          |
| 41 | // Muestra el tiempo total de ejecución                                                  |
| 42 | cout &lt;&lt; endl &lt;&lt; "Tiempo total de ejecución: " &lt;&lt; duracion.count()      |
| 43 | cout &lt;&lt; "Equivalente a: " &lt;&lt; duracion.count() / 1000.0 &lt;&lt; " seg        |
| 44 |                                                                                          |
| 45 | return 0; // Indica que el programa terminó exitosamente                                 |
| 46 | }                                                                                        |

## 1.3 Comparación de Tiempos de Ejecución

### ¿Qué tan rápido se ejecutó cada versión?

#### Resultados de ejecución:

- Python: 1.0016 segundos
- C++: 1.001 segundos (1001 milisegundos)

#### Análisis:

Ambos lenguajes muestran tiempos muy similares porque el tiempo de ejecución está dominado por las pausas de 0.1 segundos (100ms) insertadas intencionalmente. En términos de tiempo puro de procesamiento, C++ es ligeramente más rápido, pero la diferencia es mínima en este ejemplo. La diferencia real se vería en operaciones computacionalmente intensivas sin pausas artificiales, donde C++ típicamente supera a Python debido a su compilación a código nativo.

# Fase 2: Corrección de Errores

En esta fase analizaremos errores comunes en ambos lenguajes, cuándo se detectan, qué mensajes generan y cómo corregirlos.

## 2.1 Código Python con Errores (Original)

| Código | Python (con errores)                                                                             |
|--------|--------------------------------------------------------------------------------------------------|
| 1      | <code>#!/usr/bin/env python3</code>                                                              |
| 2      | <code>"""</code>                                                                                 |
| 3      | <code>Fase 2: Código Python con errores - Versión ORIGINAL (con errores)</code>                  |
| 4      | <code>Este código contiene errores intencionados para demostrar la detección y corrección</code> |
| 5      | <code>"""</code>                                                                                 |
| 6      |                                                                                                  |
| 7      | <code>import time</code>                                                                         |
| 8      |                                                                                                  |
| 9      | <code>def calcular_promedio(numeros):</code>                                                     |
| 10     | <code>    """</code>                                                                             |
| 11     | <code>    Función que calcula el promedio de una lista de números</code>                         |
| 12     | <code>    ERRORES INCLUIDOS INTENCIONALMENTE</code>                                              |
| 13     | <code>    """</code>                                                                             |
| 14     | <code>    # ERROR 1: División por cero no manejada</code>                                        |
| 15     | <code>    suma = sum(numeros)</code>                                                             |
| 16     | <code>    promedio = suma / len(numeros) # Falla si la lista está vacía</code>                   |
| 17     | <code>    return promedio</code>                                                                 |
| 18     |                                                                                                  |
| 19     | <code>def mostrar_tabla_multiplicar(numero):</code>                                              |
| 20     | <code>    """</code>                                                                             |
| 21     | <code>    Función que muestra la tabla de multiplicar</code>                                     |
| 22     | <code>    ERRORES INCLUIDOS INTENCIONALMENTE</code>                                              |
| 23     | <code>    """</code>                                                                             |
| 24     | <code>    print(f"Tabla de multiplicar del {numero}:")</code>                                    |
| 25     |                                                                                                  |
| 26     | <code>    # ERROR 2: Rango incorrecto (debería ser 1 a 10, no 0 a 9)</code>                      |
| 27     | <code>    for i in range(10): # Va de 0 a 9, no de 1 a 10</code>                                 |
| 28     | <code>        resultado = numero * i</code>                                                      |
| 29     | <code>        print(f"{numero} x {i} = {resultado}")</code>                                      |
| 30     |                                                                                                  |
| 31     | <code>def procesar_datos():</code>                                                               |
| 32     | <code>    """</code>                                                                             |
| 33     | <code>    Función principal con múltiples errores</code>                                         |

|    |                                                                                 |
|----|---------------------------------------------------------------------------------|
| 34 | """                                                                             |
| 35 | # ERROR 3: Variable no definida                                                 |
| 36 | print(f"Procesando {total_elementos} elementos...") # total_elementos no existe |
| 37 |                                                                                 |
| 38 | # ERROR 4: Lista vacía causará error en calcular_promedio                       |
| 39 | lista_vacia = []                                                                |
| 40 | promedio = calcular_promedio(lista_vacia)                                       |
| 41 | print(f"Promedio: {promedio}")                                                  |
| 42 |                                                                                 |
| 43 | # ERROR 5: Tipo de dato incorrecto                                              |
| 44 | mostrar_tabla_multiplicar("5") # String en lugar de int                         |
| 45 |                                                                                 |
| 46 | def main():                                                                     |
| 47 | """                                                                             |
| 48 | Función principal que ejecutará el código con errores                           |
| 49 | """                                                                             |
| 50 | try:                                                                            |
| 51 | procesar_datos()                                                                |
| 52 | mostrar_tabla_multiplicar(7)                                                    |
| 53 | except Exception as e:                                                          |
| 54 | print(f"Error encontrado: {e}")                                                 |
| 55 |                                                                                 |
| 56 | if __name__ == "__main__":                                                      |
| 57 | main()                                                                          |

## 2.2 Código Python Corregido

| Código | Python (corregido)                                                               |
|--------|----------------------------------------------------------------------------------|
| 1      | #!/usr/bin/env python3                                                           |
| 2      | """                                                                              |
| 3      | Fase 2: Código Python corregido - Versión CORREGIDA                              |
| 4      | Este código muestra las correcciones aplicadas a los errores del código original |
| 5      | """                                                                              |
| 6      |                                                                                  |
| 7      | import time                                                                      |
| 8      |                                                                                  |
| 9      | def calcular_promedio(numeros):                                                  |
| 10     | """                                                                              |
| 11     | Función que calcula el promedio de una lista de números                          |
| 12     | CORRIGIDO: Manejo de lista vacía                                                 |
| 13     | """                                                                              |

|    |                                                                                        |
|----|----------------------------------------------------------------------------------------|
| 14 | if not numeros: # CORRECCIÓN: Verificar si la lista está vacía                         |
| 15 | print("Error: No se puede calcular el promedio de una lista vacía")                    |
| 16 | return 0                                                                               |
| 17 |                                                                                        |
| 18 | suma = sum(numeros)                                                                    |
| 19 | promedio = suma / len(numeros)                                                         |
| 20 | return promedio                                                                        |
| 21 |                                                                                        |
| 22 | def mostrar_tabla_multiplicar(numero):                                                 |
| 23 | """                                                                                    |
| 24 | Función que muestra la tabla de multiplicar                                            |
| 25 | CORRIGIDO: Rango y validación de tipo                                                  |
| 26 | """                                                                                    |
| 27 | # CORRECCIÓN: Validar que el número sea entero                                         |
| 28 | if not isinstance(numero, int):                                                        |
| 29 | print(f"Error: El valor debe ser un número entero, recibido: {type(numero).__name__}") |
| 30 | return                                                                                 |
| 31 |                                                                                        |
| 32 | print(f"Tabla de multiplicar del {numero}:")                                           |
| 33 |                                                                                        |
| 34 | # CORRECCIÓN: Rango de 1 a 10 (inclusive)                                              |
| 35 | for i in range(1, 11): # Va de 1 a 10                                                  |
| 36 | resultado = numero * i                                                                 |
| 37 | print(f"{numero} x {i} = {resultado}")                                                 |
| 38 |                                                                                        |
| 39 | def procesar_datos():                                                                  |
| 40 | """                                                                                    |
| 41 | Función principal corregida                                                            |
| 42 | """                                                                                    |
| 43 | # CORRECCIÓN: Definir la variable antes de usarla                                      |
| 44 | total_elementos = 5 # Variable definida correctamente                                  |
| 45 | print(f"Procesando {total_elementos} elementos...")                                    |
| 46 |                                                                                        |
| 47 | # CORRECCIÓN: Usar una lista con datos en lugar de lista vacía                         |
| 48 | lista_numeros = [10, 20, 30, 40, 50] # Lista con valores                               |
| 49 | promedio = calcular_promedio(lista_numeros)                                            |
| 50 | print(f"Promedio: {promedio}")                                                         |
| 51 |                                                                                        |
| 52 | # CORRECCIÓN: Pasar un entero en lugar de string                                       |
| 53 | mostrar_tabla_multiplicar(5) # Entero en lugar de string                               |
| 54 |                                                                                        |



|    |                                                     |
|----|-----------------------------------------------------|
| 55 | def main():                                         |
| 56 | """                                                 |
| 57 | Función principal que ejecutará el código corregido |
| 58 | """                                                 |
| 59 | try:                                                |
| 60 | procesar_datos()                                    |
| 61 | mostrar_tabla_multiplicar(7)                        |
| 62 |                                                     |
| 63 | # Demostrar manejo de errores con lista vacía       |
| 64 | print("\n--- Probando con lista vacía ---")         |
| 65 | calcular_promedio([])                               |
| 66 |                                                     |
| 67 | # Demostrar manejo de errores con tipo incorrecto   |
| 68 | print("\n--- Probando con tipo incorrecto ---")     |
| 69 | mostrar_tabla_multiplicar("texto")                  |
| 70 |                                                     |
| 71 | except Exception as e:                              |
| 72 | print(f"Error inesperado: {e}")                     |
| 73 |                                                     |
| 74 | if __name__ == "__main__":                          |
| 75 | main()                                              |

## 2.3 Código C++ con Errores (Original)

| Código | C++ (con errores)                                                                     |
|--------|---------------------------------------------------------------------------------------|
| 1      | #include <iostream>                                                                   |
| 2      | #include <vector>                                                                     |
| 3      |                                                                                       |
| 4      | using namespace std;                                                                  |
| 5      |                                                                                       |
| 6      | /**                                                                                   |
| 7      | * Código C++ con errores - Versión ORIGINAL (con errores)                             |
| 8      | * Este código contiene errores intencionados para demostrar la detección y corrección |
| 9      | */                                                                                    |
| 10     |                                                                                       |
| 11     | /**                                                                                   |
| 12     | * Función que calcula el promedio de un vector de números                             |
| 13     | * ERRORES INCLUIDOS INTENCIONALMENTE                                                  |
| 14     | */                                                                                    |
| 15     | double calcularPromedio(vector<int> numeros) {                                        |
| 16     | int suma = 0;                                                                         |

|    |                                                                                 |
|----|---------------------------------------------------------------------------------|
| 17 |                                                                                 |
| 18 | // ERROR 1: No verificar si el vector está vacío (división por cero)            |
| 19 | for(int i = 0; i &lt; numeros.size(); i++) {                                    |
| 20 | suma += numeros[i];                                                             |
| 21 | }                                                                               |
| 22 |                                                                                 |
| 23 | double promedio = suma / numeros.size(); // Falla si el vector está vacío       |
| 24 | return promedio;                                                                |
| 25 | }                                                                               |
| 26 |                                                                                 |
| 27 | /**                                                                             |
| 28 | * Función que muestra la tabla de multiplicar                                   |
| 29 | * ERRORES INCLUIDOS INTENCIONALMENTE                                            |
| 30 | */                                                                              |
| 31 | void mostrarTablaMultiplicar(int numero) {                                      |
| 32 | cout &lt;&lt; "Tabla de multiplicar del " &lt;&lt; numero &lt;&lt; ":" &lt;&lt; |
| 33 |                                                                                 |
| 34 | // ERROR 2: Rango incorrecto y acceso fuera de límites                          |
| 35 | for(int i = 0; i &lt;= 10; i++) { // Va de 0 a 10, debería ser de 1 a 10        |
| 36 | int resultado = numero * i;                                                     |
| 37 | cout &lt;&lt; numero &lt;&lt; " x " &lt;&lt; i &lt;&lt; " = " &lt;&lt;          |
| 38 | }                                                                               |
| 39 | }                                                                               |
| 40 |                                                                                 |
| 41 | /**                                                                             |
| 42 | * Función que accede a elementos de un array                                    |
| 43 | * ERRORES INCLUIDOS INTENCIONALMENTE                                            |
| 44 | */                                                                              |
| 45 | void procesarArray() {                                                          |
| 46 | int numeros[5] = {1, 2, 3, 4, 5};                                               |
| 47 |                                                                                 |
| 48 | // ERROR 3: Acceso fuera de los límites del array                               |
| 49 | for(int i = 0; i &lt;= 5; i++) { // El índice 5 está fuera de límites           |
| 50 | cout &lt;&lt; "Elemento " &lt;&lt; i &lt;&lt; ": " &lt;&lt; numeros[i]          |
| 51 | }                                                                               |
| 52 | }                                                                               |
| 53 |                                                                                 |
| 54 | /**                                                                             |
| 55 | * Función principal con múltiples errores                                       |
| 56 | */                                                                              |
| 57 | int main() {                                                                    |

|    |                                                                |
|----|----------------------------------------------------------------|
| 58 | // ERROR 4: Vector vacío causará división por cero             |
| 59 | vector<int> listaVacía;                                        |
| 60 | double promedio = calcularPromedio(listaVacía);                |
| 61 | cout <<< "Promedio: " <<< promedio <<< endl;                   |
| 62 |                                                                |
| 63 | // ERROR 5: Llamar función con parámetro que causará problemas |
| 64 | mostrarTablaMultiplicar(0); // Multiplicar por 0 no es útil    |
| 65 |                                                                |
| 66 | // ERROR 6: Función que accede fuera de límites                |
| 67 | procesarArray();                                               |
| 68 |                                                                |
| 69 | return 0;                                                      |
| 70 | }                                                              |

## 2.4 Código C++ Corregido

| Código | C++ (corregido)                                                                    |
|--------|------------------------------------------------------------------------------------|
| 1      | #include <iostream>                                                                |
| 2      | #include <vector>                                                                  |
| 3      |                                                                                    |
| 4      | using namespace std;                                                               |
| 5      |                                                                                    |
| 6      | /**                                                                                |
| 7      | * Código C++ corregido - Versión CORREGIDA                                         |
| 8      | * Este código muestra las correcciones aplicadas a los errores del código original |
| 9      | */                                                                                 |
| 10     |                                                                                    |
| 11     | /**                                                                                |
| 12     | * Función que calcula el promedio de un vector de números                          |
| 13     | * CORREGIDO: Manejo de vector vacío                                                |
| 14     | */                                                                                 |
| 15     | double calcularPromedio(vector<int> numeros) {                                     |
| 16     | // CORRECCIÓN: Verificar si el vector está vacío                                   |
| 17     | if(numeros.empty()) {                                                              |
| 18     | cout <<< "Error: No se puede calcular el promedio de un vector vacío" <<< endl;    |
| 19     | return 0.0;                                                                        |
| 20     | }                                                                                  |
| 21     |                                                                                    |
| 22     | int suma = 0;                                                                      |
| 23     | for(int i = 0; i < numeros.size(); i++) {                                          |
| 24     | suma += numeros[i];                                                                |

|    |                                                                                                   |
|----|---------------------------------------------------------------------------------------------------|
| 25 | }                                                                                                 |
| 26 |                                                                                                   |
| 27 | double promedio = static_cast<double>(suma) / numeros.size(); // Cast para precisión              |
| 28 | return promedio;                                                                                  |
| 29 | }                                                                                                 |
| 30 |                                                                                                   |
| 31 | /**                                                                                               |
| 32 | * Función que muestra la tabla de multiplicar                                                     |
| 33 | * CORREGIDO: Rango y validación                                                                   |
| 34 | */                                                                                                |
| 35 | void mostrarTablaMultiplicar(int numero) {                                                        |
| 36 | // CORRECCIÓN: Validar número de entrada                                                          |
| 37 | if(numero &lt;= 0) {                                                                              |
| 38 | cout &lt;&lt;&lt; "Error: El número debe ser mayor que 0" &lt;&lt;&lt; endl;                      |
| 39 | return;                                                                                           |
| 40 | }                                                                                                 |
| 41 |                                                                                                   |
| 42 | cout &lt;&lt;&lt; "Tabla de multiplicar del " &lt;&lt;&lt; numero &lt;&lt;&lt; ":" &lt;&lt;&lt; " |
| 43 |                                                                                                   |
| 44 | // CORRECCIÓN: Rango de 1 a 10 (no incluir 0)                                                     |
| 45 | for(int i = 1; i &lt;= 10; i++) { // Va de 1 a 10                                                 |
| 46 | int resultado = numero * i;                                                                       |
| 47 | cout &lt;&lt;&lt; numero &lt;&lt;&lt; " x " &lt;&lt;&lt; i &lt;&lt;&lt; " = " &lt;&lt;&lt; "      |
| 48 | }                                                                                                 |
| 49 | }                                                                                                 |
| 50 |                                                                                                   |
| 51 | /**                                                                                               |
| 52 | * Función que accede a elementos de un array de forma segura                                      |
| 53 | * CORREGIDO: Verificación de límites                                                              |
| 54 | */                                                                                                |
| 55 | void procesarArray() {                                                                            |
| 56 | int numeros[5] = {1, 2, 3, 4, 5};                                                                 |
| 57 | int tamaño = 5; // CORRECCIÓN: Definir explícitamente el tamaño                                   |
| 58 |                                                                                                   |
| 59 | cout &lt;&lt;&lt; "Procesando array de forma segura:" &lt;&lt;&lt; endl;                          |
| 60 |                                                                                                   |
| 61 | // CORRECCIÓN: Verificar límites del array                                                        |
| 62 | for(int i = 0; i &lt; tamaño; i++) { // i &lt; tamaño, no i &lt;= tamaño                          |
| 63 | cout &lt;&lt;&lt; "Elemento " &lt;&lt;&lt; i &lt;&lt;&lt; ": " &lt;&lt;&lt; numeros[i]            |
| 64 | }                                                                                                 |
| 65 | }                                                                                                 |

|     |                                                               |
|-----|---------------------------------------------------------------|
| 66  |                                                               |
| 67  | /**                                                           |
| 68  | * Función que demuestra el manejo seguro de vectores          |
| 69  | * AÑADIDO: Nueva función para demostrar buenas prácticas      |
| 70  | */                                                            |
| 71  | void procesarVectorSeguro() {                                 |
| 72  | vector<int> numeros = {10, 20, 30, 40, 50};                   |
| 73  |                                                               |
| 74  | cout <<< "\nProcesando vector de forma segura:" <<< endl;     |
| 75  |                                                               |
| 76  | // Método seguro usando .size()                               |
| 77  | for(size_t i = 0; i < numeros.size(); i++) {                  |
| 78  | cout <<< "Elemento " << i << ": " << numeros[i]               |
| 79  | }                                                             |
| 80  |                                                               |
| 81  | // Método alternativo con iteradores (más moderno)            |
| 82  | cout <<< "\nUsando iteradores (C++ moderno):" <<< endl;       |
| 83  | for(const auto& numero : numeros) {                           |
| 84  | cout <<< "Valor: " << numero <<< endl;                        |
| 85  | }                                                             |
| 86  | }                                                             |
| 87  |                                                               |
| 88  | /**                                                           |
| 89  | * Función principal corregida                                 |
| 90  | */                                                            |
| 91  | int main() {                                                  |
| 92  | cout <<< "=== Demostrando código corregido ===" <<< endl;     |
| 93  |                                                               |
| 94  | // CORRECCIÓN: Usar vector con datos en lugar de vector vacío |
| 95  | vector<int> listaNumeros = {15, 25, 35, 45, 55};              |
| 96  | double promedio = calcularPromedio(listaNumeros);             |
| 97  | cout <<< "Promedio: " << promedio <<< endl;                   |
| 98  |                                                               |
| 99  | // CORRECCIÓN: Usar número válido para la tabla               |
| 100 | cout <<< "\n--- Tabla de multiplicar ---" <<< endl;           |
| 101 | mostrarTablaMultiplicar(7);                                   |
| 102 |                                                               |
| 103 | // CORRECCIÓN: Procesar array de forma segura                 |
| 104 | cout <<< "\n--- Procesamiento de array ---" <<< endl;         |
| 105 | procesarArray();                                              |
| 106 |                                                               |

|     |                                                                        |
|-----|------------------------------------------------------------------------|
| 107 | // AÑADIDO: Demostrar procesamiento seguro de vectores                 |
| 108 | procesarVectorSeguro();                                                |
| 109 |                                                                        |
| 110 | // Demostrar manejo de errores                                         |
| 111 | cout &lt;&lt; "\n=== Demostrando manejo de errores ===" &lt;&lt; endl; |
| 112 |                                                                        |
| 113 | // Probar con vector vacío                                             |
| 114 | vector<int> vectorVacio;                                               |
| 115 | calcularPromedio(vectorVacio);                                         |
| 116 |                                                                        |
| 117 | // Probar con número inválido                                          |
| 118 | mostrarTablaMultiplicar(0);                                            |
| 119 | mostrarTablaMultiplicar(-5);                                           |
| 120 |                                                                        |
| 121 | return 0;                                                              |
| 122 | }                                                                      |

## 2.5 Análisis de Detección de Errores

### ¿Qué pasa si hay un error en el código?

**En Python:** Los errores de sintaxis se detectan al interpretar, los errores de lógica en tiempo de ejecución. El programa se detiene y muestra un traceback detallado.

**En C++:** Los errores de sintaxis y tipos se detectan en compilación. Los errores de lógica (como división por cero) causan fallos en tiempo de ejecución o comportamiento indefinido.

### ¿Cuándo se detectan los errores en cada lenguaje?

#### Python (Interpretado):

- Errores de sintaxis: Al ejecutar la línea problemática
- Errores de nombre/tipo: En tiempo de ejecución cuando se alcanza la línea
- Ventaja: Desarrollo rápido, cambios inmediatos
- Desventaja: Errores pueden pasar desapercibidos hasta la ejecución

#### C++ (Compilado):

- Errores de sintaxis/tipos: En tiempo de compilación (antes de ejecutar)
- Errores de lógica: En tiempo de ejecución
- Ventaja: Muchos errores se detectan antes de distribuir el programa
- Desventaja: Proceso de compilación adicional

### ¿Qué mensaje de error da el compilador/intérprete?

#### Mensaje de error Python (observado):

Error encontrado: name 'total\_elementos' is not defined

#### Mensaje de error C++ (observado):

Floating point exception (core dumped)

Python proporciona mensajes más descriptivos que ayudan a identificar

exactamente qué variable o problema causó el error. C++ a menudo da mensajes más técnicos que requieren mayor experiencia para interpretar.

## ¿Cómo se pueden interpretar y corregir estos errores?

### Estrategias de corrección:

#### 1. Variables no definidas (Python):

- Leer el mensaje: "name 'variable' is not defined"
- Verificar ortografía de la variable
- Asegurar que la variable se declara antes de usarse
- Verificar el ámbito (scope) de la variable

#### 2. División por cero (C++):

- "Floating point exception" indica operación matemática inválida
- Verificar divisiones por cero
- Añadir validaciones antes de operaciones matemáticas
- Usar depuradores para encontrar la línea exacta

#### 3. Acceso fuera de límites:

- Verificar índices de arrays/listas
- Usar .size() o len() para límites
- Implementar verificaciones de rango

### Mejores prácticas:

- Validar entradas
- Manejar casos especiales (listas vacías, valores nulos)
- Usar herramientas de depuración
- Escribir código defensivo con verificaciones